EXERCISE: TIME

Create a **Time** class containing three integer attributes: ***H***, ***M***, and ***S***, and the following methods:

1. The following constructors:
   - ***Time(int hour)***: constructs a Time object from a given hour.
   - ***Time(int hour, int minute)***: constructs a Time object from a given hour and minute.
   - ***Time(int hour, int minute, int second)***: constructs a Time object from the hour, minute, and second values.
2. A ***display()*** method that displays the time in the following format: "It is h hours m minutes s seconds".
3. A ***main*** method that creates a time and displays it.

THE "CURRENT" OBJECT

- The keyword `***this***` represents a reference to the current object, that is, the object containing the method that is currently being executed.
- The `***this***` reference can be useful:
  - when a local variable (or parameter) "hides," by having the same name, an attribute of the class.
  - when a reference to the current object is not yet known.

SELF-REFERENCE: THIS

The keyword ***this*** stands for:

- clear up ambiguities
- reuse a constructor (call another constructor of the same class. This is written in the first statement of the constructor)

```
public class Point {
// …
public Point (double x, double y)
        {
                x = x;
                y = y;
        }
public Point (){
                x=0;
                y=0;
        }
}
```

ambiguity!

redundancy

```
public class Point {
// …
public Point (double x, double y)
        {
                this.x = x;
                this.y = y;
        }
public Point (){
                this(0, 0);
        }
}
```

© Soumaya Trabelsi

SYNTAX FOR DECLARING A CLASS

[**private/public**] [**abstract/final**] class NameClass
[**extends** NameSuperClass]
[**implements** NameInterface1, NameInterface2, …, NameInterfaceN]
{
    //attribute declarations
    //Methods declaration
}

© Soumaya Trabelsi

SYNTAX FOR DECLARING A CLASS

- **Private and public:** two access modifiers specifying the class's usability:
  - `private` indicates that the class is restricted to classes within the same package.
  - `public` allows all other classes to use the class.
  - By default, the modifier is `private`.
- **Abstract and final:** two qualifiers related to the concept of inheritance (to be studied later).
- **extends**: indicates inheritance from the class `SuperClassName` (to be studied later).
- **implements**: indicates the implementation of one or more interfaces by the class (to be studied later).

© Soumaya Trabelsi

DECLARATION OF ATTRIBUTES

[private/public/protected] **[static]** **[final]** *type_attribut* **name_attribut** [=value];

- **Private, public, and protected**: modifiers for access to the attribute from other classes.
  - **Private**: indicates that access is restricted to methods of the same class.
  - **Public**: allows access to all methods of other classes.
  - **Protected**: limits access to methods of the same class, derived classes, and classes in the same package.
  - By default, access is friendly, meaning it is private to the package (not private to the class).
- **Static**: specifies that there will be only one copy of this attribute, common to all objects created from the class in question (it is a class attribute).
- **Final**: indicates that the attribute is a constant.

© Soumaya Trabelsi

DECLARATION OF METHODS

[private/public/protected] **[static]** **[final/abstract]** *type_return* name_method ([type_parameter1 parameter1, …, type_parameterN parameterN])

{

// local variable declarations;

[*type_return* var_result;]

//instructions for the method

[return var_results;]

}

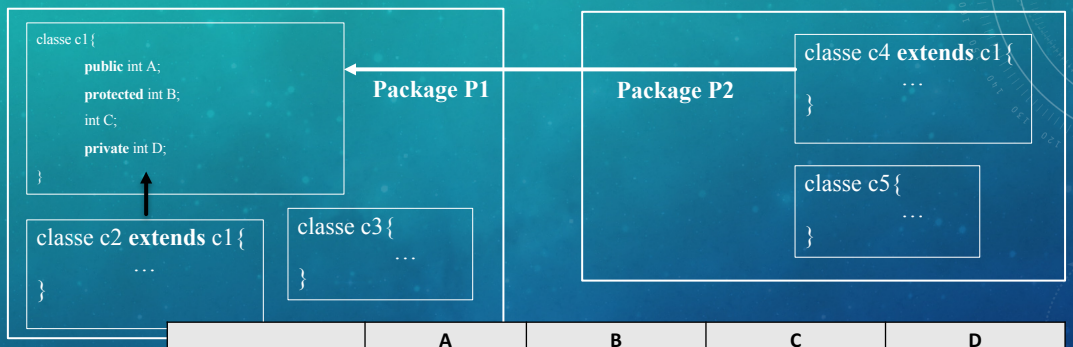
© Soumaya Trabelsi

DECLARATION OF METHODS

- **private, public, and protected**: access modifiers that allow calls to the class's methods (see attribute declarations).
- **static**: specifies that the method's execution is not tied to instances of the class (it is a class method).
- **abstract**: the method has no code. Therefore, any class becomes abstract (to be studied later).
- **final**: the method cannot be redefined in a derived class (to be studied later).

© Soumaya Trabelsi

VISIBILITY OF CLASS MEMBERS



	A	B	C	D
Accessible by c2	x	x	x	-
Accessible by c3	x	x	x	-
Accessible by c4	x	x	-	-
Accessible by c5	x	-	-	-

© Soumaya Trabelsi

VISIBILITY OF CLASS MEMBERS

Accessible to:	Methods of the same class	Derived classes in the same package	Classes in the same package	Derived classes in another package	Classes from other packages
public	x	x	x	x	x
protected	x	x	x	x	
friendly	x	x	x		
private	x				

© Soumaya Trabelsi

SCOPE OF VARIABLES

- Variables are known and not known except within the block in which they are declared.

Exemple:

```
public class AClass {
  int i, j, k;
  public void Sth1(int z){
    int j,r;
    r=z;
   }
  public void Sth2(){
    k=r;
   }
}
```

j: These are two different variables

k is known at the level of the *Sth2()* method because it is declared in the class block. However, **r** is not defined for *Sth2()*. This results in a compilation error.

© Soumaya Trabelsi

SCOPE OF VARIABLES

In case of a name conflict between local and global variables, the most local variable is always considered to be the variable designated by that part of the program.

```
public class AClass {
  int i, j, k;
  public void Sth1 (int z){
    int j,r;
    j=z;
   }
}
```

© Soumaya Trabelsi

METHOD OVERLOADING

Definition: Writing multiple methods with the same name in a class.

- The methods must differ in the number of arguments and/or the type of those arguments.
- The return type can also be different, but not exclusively (i.e., it cannot be the only difference)!
- Having multiple constructors is a special case of overloading.
- Method overloading is called static polymorphism.

© Soumaya Trabelsi

METHOD OVERLOADING

Example:

```
int solde;
public void deposit (int amount){
    solde+= amount;
}

public void deposit (double amount){
solde+= (int) amount;
}
```

© Soumaya Trabelsi

ENCAPSULATION

- Encapsulation is a mechanism for bringing together data and methods within a structure (the class).
- Implementation details are hidden, with the outside world only having access to the data through a set of operations constituting the object's interface.
- Encapsulation therefore makes it possible to guarantee the integrity of the data contained in the object.

EXAMPLE ON ACCESS CONTROL

```
public class Parallelogramme{
    private int longueur = 0; //declaration + explicit initialization
    private int largeur = 0; // declaration + explicit initialization
    public int profondeur = 0; //declaration + explicit initialization
    public void affiche () {
    System.out.println("longueur = "+ longueur + "Largeur =  " +largeur + "Profondeur = " +      profondeur);
    }
}
public class ProgParal {
        public static void main (String args {})
        {
                Parallelogramme p1 =  new Parallellogramme ();
                p1.longueur = 5; // invalid because the attribute is private
                p1.profondeur = 4; // OK
                p1.affiche(); // OK
}}
```